

Assignment 3: Handling Events and Forms

Subject: ReactJS | Unit 3: Handling Events and Forms

B.Sc. Computer Science

Q1. Handle Button Click Event and Display Alert

onClick is the most commonly used event in React. When a button is clicked, React calls the specified event handler function. The handler receives a SyntheticEvent object. Here we display an alert on button click.

Program / Code:

```
import React, { useState } from 'react';

function ButtonClick() {
  const [message, setMessage] = useState('No button clicked yet.');
```



```
  const handleClick = () => {
    alert('Hello! Button was clicked!');
    setMessage('Button was clicked at: ' + new Date().toLocaleTimeString());
  };

  const handleDoubleClick = () => {
    setMessage('Double click detected!');
  };

  return (
    <div>
      <h2>Button Click Event Demo</h2>
      <p>{message}</p>
      <button onClick={handleClick}>Click Me</button>
      <button onDoubleClick={handleDoubleClick}>Double Click Me</button>
    </div>
  );
}
```



```
export default ButtonClick;
```

Expected Output:

```
Output (initial):      No button clicked yet.
Output (single click): Alert: 'Hello! Button was clicked!'
                        Button was clicked at: 10:30:45 AM
Output (double click): Double click detected!
```

Q2. Event Handling in React

React uses SyntheticEvents — a cross-browser wrapper around native browser events. React events use camelCase (onClick, onChange, onMouseOver) unlike HTML (onclick). Below we demonstrate multiple event types in one component.

Program / Code:

```
import React, { useState } from 'react';

function EventHandling() {
  const [log, setLog] = useState([]);

  const addLog = (msg) => setLog(prev => [...prev, msg]);

  return (
    <div>
      <h2>React Event Handling Demo</h2>

      <button onClick={() => addLog('onClick fired!')}
        onMouseEnter={() => addLog('onMouseEnter fired!')}
        onMouseLeave={() => addLog('onMouseLeave fired!')}>
        Hover & Click Me
      </button>

      <input
        type='text'
        onChange={(e) => addLog('onChange: ' + e.target.value)}
        onFocus={() => addLog('onFocus: input focused')}
        onBlur={() => addLog('onBlur: input lost focus')}
        placeholder='Type here...'
      />

      <h4>Event Log:</h4>
      <ul>
        {log.slice(-5).map((entry, i) => <li key={i}>{entry}</li>)}
      </ul>
    </div>
  );
}

export default EventHandling;
```

Expected Output:

```
Output (on hover): onMouseEnter fired!
Output (on click): onClick fired!
Output (on type):  onChange: H
                  onChange: He
Output (on focus): onFocus: input focused
Output (on blur):  onBlur: input lost focus
```

Q3. Controlled Form Component

In a Controlled Component, React state is the single source of truth for input values. Every keystroke updates state via onChange, and the input's value is always set from state. This gives React full control over the form.

Program / Code:

```
import React, { useState } from 'react';

function ControlledForm() {
  const [name, setName] = useState('');
  const [email, setEmail] = useState('');
  const [submitted, setSubmitted] = useState(false);

  const handleSubmit = (e) => {
    e.preventDefault();
    setSubmitted(true);
  };

  return (
    <div>
      <h2>Controlled Form Component</h2>
      <form onSubmit={handleSubmit}>
        <div>
          <label>Name: </label>
          <input
            type='text'
            value={name}
            onChange={(e) => setName(e.target.value)}
            placeholder='Enter name'
          />
        </div>
        <div>
          <label>Email: </label>
          <input
            type='email'
            value={email}
            onChange={(e) => setEmail(e.target.value)}
            placeholder='Enter email'
          />
        </div>
        <button type='submit'>Submit</button>
      </form>
      {submitted && (
        <p>Submitted - Name: {name}, Email: {email}</p>
      )}
    </div>
  );
}

export default ControlledForm;
```

Expected Output:

```
Output (typing):   Name: [Rahul      ] Email: [rahul@mail.com]
                  [ Submit ]
Output (submit):   Submitted - Name: Rahul, Email: rahul@mail.com
```

Q4. Uncontrolled Form Component using Refs

In an Uncontrolled Component, the form data is handled by the DOM itself, not by React state. We use `useRef()` to access input values only when needed (e.g., on form submission), rather than on every keystroke.

Program / Code:

```
import React, { useRef, useState } from 'react';

function UncontrolledForm() {
  const nameRef = useRef(null);
  const emailRef = useRef(null);
  const [result, setResult] = useState('');

  const handleSubmit = (e) => {
    e.preventDefault();
    // Read values from DOM via refs
    const name = nameRef.current.value;
    const email = emailRef.current.value;
    setResult(`Name: ${name} | Email: ${email}`);
  };

  return (
    <div>
      <h2>Uncontrolled Form using Refs</h2>
      <form onSubmit={handleSubmit}>
        <div>
          <label>Name: </label>
          <input ref={nameRef} type='text'
            defaultValue='Rahul' />
        </div>
        <div>
          <label>Email: </label>
          <input ref={emailRef} type='email'
            defaultValue='rahul@example.com' />
        </div>
        <button type='submit'>Get Values</button>
      </form>
      {result && <p><b>Result:</b> {result}</p>}
    </div>
  );
}

export default UncontrolledForm;
```

Expected Output:

```
Output (initial):   Name: [Rahul          ] Email: [rahul@example.com]
                   [ Get Values ]
Output (on submit): Result: Name: Rahul | Email: rahul@example.com
Note: Values read from DOM directly via refs, not from React state.
```

Q5. Login Form with Username and Password

A login form captures username and password from the user. We use controlled components for both fields, validate that neither is empty, and show appropriate messages on submission.

Program / Code:

```
import React, { useState } from 'react';

function LoginForm() {
  const [username, setUsername] = useState('');
  const [password, setPassword] = useState('');
  const [message, setMessage] = useState('');

  const handleLogin = (e) => {
    e.preventDefault();
    if (!username || !password) {
      setMessage('Please fill in all fields!');
      return;
    }
    // Simple demo validation
    if (username === 'admin' && password === '1234') {
      setMessage('Login Successful! Welcome, ' + username);
    } else {
      setMessage('Invalid username or password!');
    }
  };

  return (
    <div style={{ maxWidth:'300px', margin:'20px auto' }}>
      <h2>Login Form</h2>
      <form onSubmit={handleLogin}>
        <div>
          <label>Username:</label><br />
          <input type='text' value={username}
            onChange={(e) => setUsername(e.target.value)}
            placeholder='Enter username' />
        </div><br />
        <div>
          <label>Password:</label><br />
          <input type='password' value={password}
            onChange={(e) => setPassword(e.target.value)}
            placeholder='Enter password' />
        </div><br />
        <button type='submit'>Login</button>
      </form>
      {message && <p>{message}</p>}
    </div>
  );
}

export default LoginForm;
```

Expected Output:

```
Output (empty submit):   Please fill in all fields!
Output (wrong creds):    Invalid username or password!
Output (admin / 1234):   Login Successful! Welcome, admin
```

Q6. Handle Form Submission Event

The onSubmit event fires when a form is submitted. We must call e.preventDefault() to stop the default browser page-reload behavior. Below we collect multiple fields and display all submitted data.

Program / Code:

```
import React, { useState } from 'react';

function FormSubmission() {
  const [form, setForm] = useState({
    name: '', course: '', year: ''
  });
  const [submitted, setSubmitted] = useState(null);

  const handleChange = (e) => {
    setForm({ ...form, [e.target.name]: e.target.value });
  };

  const handleSubmit = (e) => {
    e.preventDefault(); // Prevent page reload
    setSubmitted({ ...form });
    setForm({ name: '', course: '', year: '' }); // Reset form
  };

  return (
    <div>
      <h2>Form Submission Demo</h2>
      <form onSubmit={handleSubmit}>
        <input name='name' value={form.name}
          onChange={handleChange} placeholder='Your Name' /><br />
        <input name='course' value={form.course}
          onChange={handleChange} placeholder='Course' /><br />
        <input name='year' value={form.year}
          onChange={handleChange} placeholder='Year' /><br />
        <button type='submit'>Submit</button>
      </form>
      {submitted && (
        <div>
          <h3>Submitted Data:</h3>
          <p>Name: {submitted.name}</p>
          <p>Course: {submitted.course}</p>
          <p>Year: {submitted.year}</p>
        </div>
      )}
    </div>
  );
}

export default FormSubmission;
```

Expected Output:

```
Output (after filling and submitting):
Submitted Data:
Name    : Rahul Patil
Course  : B.Sc. Computer Science
Year    : Third Year
(Form fields reset to empty after submission)
```

Q7. Form Validation – Name, Email, Password

Form validation checks user input before submission. We validate: name is not empty, email matches a regex pattern, and password is at least 6 characters. Errors are shown inline.

Program / Code:

```

import React, { useState } from 'react';

function FormValidation() {
  const [form, setForm] = useState({ name:'', email:'', password:'' });
  const [errors, setErrors] = useState({});

  const validate = () => {
    let e = {};
    if (!form.name.trim())
      e.name = 'Name is required.';
    if (!form.email.match(/^[\\w.-]+@[\\w.-]+\\.\\w+$/))
      e.email = 'Enter a valid email address.';
    if (form.password.length < 6)
      e.password = 'Password must be at least 6 characters.';
    return e;
  };

  const handleSubmit = (e) => {
    e.preventDefault();
    const errs = validate();
    if (Object.keys(errs).length > 0) {
      setErrors(errs);
    } else {
      setErrors({});
      alert('Form submitted successfully!');
    }
  };

  const handleChange = (e) => {
    setForm({ ...form, [e.target.name]: e.target.value });
  };

  return (
    <div>
      <h2>Form Validation Demo</h2>
      <form onSubmit={handleSubmit}>
        <div>
          <input name='name' value={form.name}
            onChange={handleChange} placeholder='Name' />
          {errors.name && <span style={{color:'red'}}> {errors.name}</span>}
        </div>
        <div>
          <input name='email' value={form.email}
            onChange={handleChange} placeholder='Email' />
          {errors.email && <span style={{color:'red'}}> {errors.email}</span>}
        </div>
        <div>
          <input name='password' type='password' value={form.password}
            onChange={handleChange} placeholder='Password' />
          {errors.password &&
            <span style={{color:'red'}}> {errors.password}</span>}
        </div>
        <button type='submit'>Register</button>
      </form>
    </div>
  );
}

export default FormValidation;

```

Expected Output:


```
Output (all empty):      Name is required.  
                        Enter a valid email address.  
                        Password must be at least 6 characters.  
Output (valid data):    Alert: Form submitted successfully!
```

Q8. Set Default Values in Input Fields

In controlled components, default values are set via `useState()` initial value. In uncontrolled components, use the `defaultValue` attribute. For checkboxes and radios, use `defaultChecked`. Below we show both approaches.

Program / Code:

```

import React, { useState } from 'react';

function DefaultValues() {
  // Controlled: default values set in useState
  const [name, setName] = useState('Rahul Patil');
  const [city, setCity] = useState('Solapur');
  const [gender, setGender] = useState('male');
  const [agree, setAgree] = useState(true);
  const [course, setCourse] = useState('BSc');

  return (
    <div>
      <h2>Default Values in Form Fields</h2>

      <label>Name: </label>
      <input value={name} onChange={e => setName(e.target.value)} />

      <br />
      <label>City: </label>
      <input value={city} onChange={e => setCity(e.target.value)} />

      <br />
      <label>Gender: </label>
      <input type='radio' value='male' checked={gender==='male'}
        onChange={() => setGender('male')} /> Male
      <input type='radio' value='female' checked={gender==='female'}
        onChange={() => setGender('female')} /> Female

      <br />
      <label>Course: </label>
      <select value={course} onChange={e => setCourse(e.target.value)}>
        <option value='BSc'>B.Sc.</option>
        <option value='BCA'>BCA</option>
        <option value='MCA'>MCA</option>
      </select>

      <br />
      <input type='checkbox' checked={agree}
        onChange={e => setAgree(e.target.checked)} />
      <label> I agree to terms</label>

      <br /><br />
      <p>Name: {name} | City: {city} | Gender: {gender}</p>
      <p>Course: {course} | Agree: {agree ? 'Yes' : 'No'}</p>
    </div>
  );
}

export default DefaultValues;

```

Expected Output:

Output (initial defaults):

Name: [Rahul Patil] City: [Solapur]

Gender: (o) Male () Female

Course: [B.Sc. v]

[x] I agree to terms

Name: Rahul Patil | City: Solapur | Gender: male | Agree: Yes

Q9. Registration Form using Controlled Components

A registration form collects full user details. Using controlled components, every field is tied to React state. We handle all input types: text, email, password, radio, select, and checkbox in a single form.

Program / Code:

```
import React, { useState } from 'react';

function RegistrationForm() {
  const [form, setForm] = useState({
    fullName: '', email: '', password: '',
    gender: 'male', course: 'BSc', terms: false
  });
  const [msg, setMsg] = useState('');

  const handleChange = (e) => {
    const { name, value, type, checked } = e.target;
    setForm({ ...form, [name]: type === 'checkbox' ? checked : value });
  };

  const handleSubmit = (e) => {
    e.preventDefault();
    if (!form.terms) { setMsg('Please accept terms!'); return; }
    setMsg('Registered: ' + form.fullName + ' (' + form.email + ')');
  };

  return (
    <div>
      <h2>Registration Form</h2>
      <form onSubmit={handleSubmit}>
        <input name='fullName' value={form.fullName}
          onChange={handleChange} placeholder='Full Name' required /><br />
        <input name='email' type='email' value={form.email}
          onChange={handleChange} placeholder='Email' required /><br />
        <input name='password' type='password' value={form.password}
          onChange={handleChange} placeholder='Password' required /><br />
        <label>Gender: </label>
        <input type='radio' name='gender' value='male'
          checked={form.gender==='male'} onChange={handleChange} /> Male
        <input type='radio' name='gender' value='female'
          checked={form.gender==='female'} onChange={handleChange} /> Female
        <br />
        <select name='course' value={form.course} onChange={handleChange}>
          <option value='BSc'>B.Sc. CS</option>
          <option value='BCA'>BCA</option>
          <option value='MCA'>MCA</option>
        </select><br />
        <input type='checkbox' name='terms'
          checked={form.terms} onChange={handleChange} />
        <label> Accept Terms & Conditions</label><br />
        <button type='submit'>Register</button>
      </form>
      {msg && <p>{msg}</p>}
    </div>
  );
}

export default RegistrationForm;
```

Expected Output:

Output (without terms): Please accept terms!
Output (with valid data + terms checked):
Registered: Rahul Patil (rahul@example.com)

Q10. Handle Key Events in React

React supports keyboard events: `onKeyDown`, `onKeyUp`, and `onKeyPress`. `onKeyDown` fires when a key is pressed down. We can detect specific keys using `e.key` (e.g., 'Enter', 'Escape', 'ArrowUp') or `e.keyCode`.

Program / Code:

```
import React, { useState } from 'react';

function KeyEvents() {
  const [log, setLog] = useState([]);
  const [input, setInput] = useState('');

  const handleKeyDown = (e) => {
    const entry = `KeyDown: key='${e.key}' code='${e.code}'`;
    setLog(prev => [entry, ...prev.slice(0, 4)]);
  };

  const handleKeyUp = (e) => {
    if (e.key === 'Enter') {
      setLog(prev => ['Enter pressed - submitted!', ...prev.slice(0,4)]);
      setInput('');
    }
    if (e.key === 'Escape') {
      setInput('');
      setLog(prev => ['Escape - input cleared!', ...prev.slice(0,4)]);
    }
  };

  return (
    <div>
      <h2>Key Events Demo</h2>
      <input
        value={input}
        onChange={e => setInput(e.target.value)}
        onKeyDown={handleKeyDown}
        onKeyUp={handleKeyUp}
        placeholder='Type and press Enter or Escape'
      />
      <h4>Key Log (last 5):</h4>
      <ul>
        {log.map((entry, i) => <li key={i}>{entry}</li>)}
      </ul>
    </div>
  );
}

export default KeyEvents;
```

Expected Output:

```
Output (typing 'Hi' then Enter):  
KeyDown: key='H'      code='KeyH'  
KeyDown: key='i'      code='KeyI'  
KeyDown: key='Enter' code='Enter'  
Enter pressed - submitted! <- input cleared  
(pressing Escape clears the input field)
```

Q11. Capture Keyboard Input using onKeyPress

onKeyPress fires when a key that produces a character value is pressed. Note: onKeyPress is deprecated in modern browsers — onKeyDown is preferred. Below we capture and display each character typed in real time.

Program / Code:

```
import React, { useState } from 'react';

function KeyPressCapture() {
  const [typed, setTyped] = useState('');
  const [keyList, setKeyList] = useState([]);
  const [count, setCount] = useState(0);

  const handleKeyPress = (e) => {
    // onKeyPress gives the character pressed
    setKeyList(prev => [...prev, e.key]);
    setCount(prev => prev + 1);
  };

  const handleChange = (e) => {
    setTyped(e.target.value);
  };

  return (
    <div>
      <h2>Keyboard Input Capture</h2>
      <input
        type='text'
        value={typed}
        onChange={handleChange}
        onKeyPress={handleKeyPress}
        placeholder='Start typing...'
        style={{ padding: '8px', width: '250px' }}
      />
      <p>You typed: <b>{typed}</b></p>
      <p>Total keys pressed: <b>{count}</b></p>
      <p>Keys: {keyList.join(', ')}</p>
      <button onClick={() => { setTyped(''); setKeyList([]); setCount(0); }}>
        Clear
      </button>
    </div>
  );
}

export default KeyPressCapture;
```

Expected Output:

```
Output (typing 'React'):  
You typed: React  
Total keys pressed: 5  
Keys: R, e, a, c, t
```

Q12. Form with Multiple Input Fields – Display Entered Data

Using a single state object for all form fields is a clean pattern. We use a computed property name [e.target.name] to update the correct field dynamically. All entered data is displayed live as a summary.

Program / Code:

```
import React, { useState } from 'react';

function MultiFieldForm() {
  const [data, setData] = useState({
    firstName: '', lastName: '', mobile: '',
    address: '', pincode: '', hobby: ''
  });

  const handleChange = (e) => {
    setData({ ...data, [e.target.name]: e.target.value });
  };

  const fields = [
    { name:'firstName', label:'First Name', type:'text' },
    { name:'lastName', label:'Last Name', type:'text' },
    { name:'mobile', label:'Mobile No.', type:'tel' },
    { name:'address', label:'Address', type:'text' },
    { name:'pincode', label:'Pin Code', type:'number' },
    { name:'hobby', label:'Hobby', type:'text' },
  ];

  return (
    <div>
      <h2>Multi-Field Form</h2>
      {fields.map(f => (
        <div key={f.name}>
          <label>{f.label}: </label>
          <input name={f.name} type={f.type}
            value={data[f.name]} onChange={handleChange}
            placeholder={`Enter ${f.label}`} />
        </div>
      ))}
      <hr />
      <h3>Live Preview:</h3>
      {fields.map(f => (
        <p key={f.name}><b>{f.label}</b> {data[f.name] || '-'}</p>
      ))}
    </div>
  );
}

export default MultiFieldForm;
```

Expected Output:

Output (after filling):

First Name : Rahul
Last Name : Patil
Mobile No. : 9876543210
Address : MG Road, Solapur
Pin Code : 413001
Hobby : Coding

Q13. Synthetic Events in React

React wraps native browser events in a `SyntheticEvent` object to ensure consistent behavior across all browsers. It has the same interface as native events (`target`, `preventDefault`, `stopPropagation`). Below we inspect the event object.

Program / Code:

```
import React, { useState } from 'react';

function SyntheticEventDemo() {
  const [info, setInfo] = useState({});

  const handleClick = (e) => {
    setInfo({
      type:      e.type,
      target:    e.target.tagName,
      bubbles:   String(e.bubbles),
      cancelable: String(e.cancelable),
      timeStamp: e.timeStamp.toFixed(2),
      nativeType: e.nativeEvent.type
    });
  };

  const handleInput = (e) => {
    setInfo({
      type: e.type,
      value: e.target.value,
      name: e.target.name,
    });
  };

  return (
    <div>
      <h2>Synthetic Events Demo</h2>
      <button onClick={handleClick}>Click to inspect SyntheticEvent</button>
      <input name='demo' onChange={handleInput}
        placeholder='Type to inspect onChange event' />
      <h4>SyntheticEvent Properties:</h4>
      <ul>
        {Object.entries(info).map(([k,v]) => (
          <li key={k}><b>{k}</b> {v}</li>
        ))}
      </ul>
    </div>
  );
}

export default SyntheticEventDemo;
```

Expected Output:

Output (after button click):

```
type      : click
target    : BUTTON
bubbles   : true
cancelable : true
timeStamp : 12453.20
nativeType : click
```


Q14. Inline CSS Styles in React Component

In React, inline styles are written as JavaScript objects with camelCase property names (backgroundColor instead of background-color). The style attribute accepts an object, not a string like in HTML.

Program / Code:

```
import React, { useState } from 'react';

function InlineStyles() {
  const [theme, setTheme] = useState('light');

  const containerStyle = {
    padding: '20px',
    background: theme === 'dark' ? '#1e1e2e' : '#f5f5f5',
    color: theme === 'dark' ? 'ffffff' : '#333333',
    borderRadius: '8px',
    transition: 'all 0.3s ease'
  };

  const headingStyle = {
    fontSize: '24px',
    fontWeight: 'bold',
    color: theme === 'dark' ? '#82b1ff' : '#1a237e',
    marginBottom: '10px'
  };

  const btnStyle = {
    padding: '8px 16px',
    background: '#3949ab',
    color: 'fff',
    border: 'none',
    borderRadius: '4px',
    cursor: 'pointer',
    marginTop: '10px'
  };

  return (
    <div style={containerStyle}>
      <h2 style={headingStyle}>Inline CSS Styles Demo</h2>
      <p style={{ fontSize: '14px', letterSpacing: '0.5px' }}>
        Current Theme: <b>{theme}</b>
      </p>
      <p style={{ color: '#e53935', fontStyle: 'italic' }}>
        Red italic text using inline style.
      </p>
      <button style={btnStyle}
        onClick={() => setTheme(theme==='light'? 'dark': 'light')}>
        Toggle Theme
      </button>
    </div>
  );
}

export default InlineStyles;
```

Expected Output:

Output (light theme):

[Light background container]

Inline CSS Styles Demo (in dark blue)

Current Theme: light

Red italic text using inline style.

[Toggle Theme]

Output (dark theme): background changes to #1e1e2e, text turns white

Q15. Simple Animation using CSS in React Component

CSS animations in React can be applied via className (external CSS) or via a style tag injected in JSX. We use @keyframes animations defined inside a style block to create bounce, spin, pulse and fade effects.

Program / Code:

```
import React, { useState } from 'react';

function AnimationDemo() {
  const [active, setActive] = useState('bounce');

  const animations = ['bounce', 'spin', 'pulse', 'fadeIn'];

  return (
    <div>
      {/* Inject keyframe CSS */}
      <style>{`
        @keyframes bounce {
          0%,100% { transform: translateY(0); }
          50%    { transform: translateY(-30px); }
        }
        @keyframes spin {
          from { transform: rotate(0deg); }
          to   { transform: rotate(360deg); }
        }
        @keyframes pulse {
          0%,100% { transform: scale(1); }
          50%    { transform: scale(1.3); }
        }
        @keyframes fadeIn {
          from { opacity: 0; }
          to   { opacity: 1; }
        }
        .anim-box {
          width: 80px; height: 80px; background: #3949ab;
          borderRadius: 8px; margin: 20px auto;
          animation-duration: 1s;
          animation-iteration-count: infinite;
        }
      `}</style>

      <h2>CSS Animation Demo</h2>
      <div>
        className='anim-box'
        style={{ animationName: active }}
      />

      <p>Active: <b>{active}</b></p>
      {animations.map(a => (
        <button key={a} onClick={() => setActive(a)}
          style={{ margin: '4px', background: active===a?'#1a237e':'#3949ab',
            color: 'white', border: 'none', padding: '6px 12px' }}>
          {a}
        </button>
      ))}
    </div>
  );
}

export default AnimationDemo;
```

Expected Output:

Output:

[Blue animated box playing 'bounce' animation]

Active: bounce

[bounce] [spin] [pulse] [fadeIn]

Clicking each button switches the animation on the box.

bounce: box moves up and down

spin: box rotates 360 degrees

pulse: box scales in and out

fadeIn: box fades from invisible to visible

End of Assignment 3 — Handling Events and Forms Topics: onClick, onChange, onKeyDown, onKeyPress, Controlled & Uncontrolled Forms, Form Validation, Default Values, Synthetic Events, Inline CSS, CSS Animations.